Título de tu Informe

Tu Nombre

8 de octubre de 2026

1. Introduccion

Seguimos con la misma situación planteada en el trabajo práctico anterior, pero ahora pasaron varios años. Mateo ahora tiene 7 años. Los mismos años que tenía Sophia cuando comenzaron a jugar al juego de las monedas. Eso quiere decir que Mateo también ya aprendió sobre algoritmos greedy, y lo comenzó a aplicar. Esto hace que ahora quién gane dependa más de quién comience y un tanto de suerte.

Esto no le gusta nada a Sophia. Ella quiere estar segura de ganar siempre. Lo bueno es que ella comenzó a aprender sobre programación dinámica. Ahora va a aplicar esta nueva técnica para asegurarse ganar siempre que pueda.

1.1. Consigna

1. **[Obligatorio]** Hacer un análisis del problema, plantear la ecuación de recurrencia correspondiente y proponer un algoritmo por programación dinámica que obtenga la solución óptima al problema planteado: Dada la secuencia de monedas $m_1, m_2, \ldots, m_n$, sabiendo que Sophia empieza el juego y que Mateo siempre elegirá la moneda más grande para sí entre la primera y la última moneda en sus respectivos turnos, definir qué monedas debe elegir Sophia para asegurarse obtener el **máximo valor acumulado posible**. Esto no necesariamente le asegurará a Sophia ganar, ya que puede ser que esto no sea posible, dado por cómo juega Mateo. Por ejemplo, para $[1, 10, 5]$, no importa lo que haga Sophia, Mateo ganará.

2. **[Obligatorio]** Demostrar que la ecuación de recurrencia planteada en el punto anterior en efecto nos lleva a obtener el **máximo valor acumulado posible**.

3. **[Obligatorio]** Escribir el algoritmo planteado. Describir y justificar la complejidad de dicho algoritmo.

4. **[Opcional]** Analizar si (y cómo) afecta a los tiempos del algoritmo planteado la variabilidad de los valores de las monedas. Como caso particular, considerar casos en los que en el TP1 el algoritmo Greedy no permitía que Sophia gane (si que empate) cuando había alguna forma para que ella pudiera ganar.

5. **[Opcional]** Realizar ejemplos de ejecución para encontrar soluciones y corroborar lo encontrado. Adicionalmente, el curso proveerá con algunos casos particulares que deben cumplirse su optimalidad también.

6. **[Opcional]** De las pruebas anteriores, hacer también mediciones de tiempos para corroborar la complejidad teórica indicada. Realizar gráficos correspondientes. Generar todo set de datos necesarios para estas pruebas.

7. **[Opcional]** Agregar cualquier conclusión que parezca relevante.

2. Algoritmo para elegir las monedas

En este trabajo de ejemplo realizaremos el análisis teórico y empírico de diferentes algoritmos que sirven para resolver el mismo problema: obtener el orden en que se deben elegir las monedas para maximizar la ganancia obtenida de un arreglo de n monedas.

2.1. Algoritmo para encontrar el optimo

El algoritmo plantea una matriz de $N \times N$, siendo n la cantidad de monedas, donde para cada coordenada i, j de la matriz, encontramos una solucion para un problema de dimensiones i, j. Luego se llena las diagonales, donde las corrdenads i y j son iguales, $i = j$, y la diagonal donde la coordenada j es uno mas que la i, $i = j + 1$, con los valores de los dos casos base, siendo estos:

- tener una sola mondea, donde el optimo es la moneda en si.
- tener dos monedas, donde el optimo es la mas grande entre la dos.

Luego recorre la parte superior de la matriz (ya que los punteros i, j que apuntan al principio y final del array de monedas respectivametne, nunca se intercambian, lo que es igual, se cummple que $i <= j$), llenando cada lugar de la matriz con la solucion del problema de tamaño i, j. Es asi que una vez completamos la matriz, encontramos la solucion optima para nuestro problema original en la coordenada $i = 0$ y $j = n$.

Aqui el codigo usado para generar la matriz con los valores optimos para todos los problemas de dimensiones i, j.

```
def elegir(monedas):
    opt = [[0] * len(monedas) for i in range(len(monedas))]

    # Casos base con una moneda
    for i in range(0, len(monedas)):
        opt[i][i] = monedas[i]

    # Casos base con 2 monedas
    for i in range(0, len(monedas) - 1):
        j = i + 1
        opt[i][j] = max(monedas[i], monedas[j])

    for i in range(len(opt) - 1, -1, -1):
        for j in range(i + 2, len(opt)):
            # Sophia toma la primera
            # Achico mi rango segun lo que eliga Mateo
            opt_i = opt[i + 2][j] if monedas[i + 1] > monedas[j]
    else opt[i + 1][j - 1]

            # Sophia toma la ultima
            # Achico mi rango segun lo que eliga Mateo
            opt_j = opt[i + 1][j - 1] if monedas[i] > monedas[j -
    1] else opt[i][j - 2]
```

```
22
23              opt[i][j] = max(monedas[i] + opt_i, monedas[j] + opt_j)
24
25      maxima_ganancia = opt[0][len(opt) - 1]
26      sol = solucion(opt, monedas, [], 0, len(opt) - 1)
27      return sol
```

La forma de obtener nuesta matriz optima es la siguente. Se plantea que para cada problema de dimensiones i, j, donde queremos encontrar que moneda elegir para mazimizar la ganancia, la moneda en la posicion i (la primera), y la moneda j (la ultima), se puden analizar las siguentes opciones.

1. Sophia toma la primera moneda, la moneda i y Mateo elige la mayor entre las monedas $i+1$ y j.

   a) Si mateo eligie la moneda $i+1$ el rango (i, j) se achica a $(i+2, j)$, por lo que nuestra solucion sera el valor de la moneda i mas la solucion del rango $(i+2, j)$.
   $opt(i, j) = moneda_i + opt(i+2, j)$

   b) Si mateo eligie la moneda j el rango (i, j) se achica a $(i+1, j-1)$, por lo que nuestra solucion sera el valor de la moneda i mas la solucion del rango $(i+1, j-j)$.
   $opt(i, j) = moneda_i + opt(i+1, j-1)$

2. Sophia toma ahora la ultima moneda, la moneda j y Mateo elige la mayor entre las monedas i y $j-1$.

   a) Si mateo eligio la moneda i el rango (i, j) se achica a $(i+1, j-1)$, por lo que nuestra solucion sera el valor de la moneda j mas la solucion del rango $(i+1, j-1)$.
   $opt(i, j) = moneda_j + opt(i+1, j-1)$

   b) Si mateo eligio la moneda j el rango (i, j) se achica a $(i, j-2)$, por lo que nuestra solucion sera el valor de la moneda j mas la solucion del rango $(i, j-2)$.
   $opt(i, j) = moneda_i + opt(i, j-2)$

Luego de plantear las cuatro posibles situaciones, podemos ver que, sin importar lo que elija sophia, mateo siempre elije la moneda mas grande por lo que en ambos casos en simplemente realizar una comparacion para ver cual es mas grande, segun la situacion de monedas. Esta comparacion determina el rango de mis soluciones parciales. Luego obtengo mis soluciones parciales para los rangos respectivos segun la eleccion de mateo, me quedo con aquella solucion, que maximize mi ganancia. Entonces Siendo opt_i, la solucion para un rango (i_i, j_i), asumiendo que sophia toma la $monedai$, y, siendo opt_j, la solucion para un rango (i_j, j_j), asumiendo que sophia toma la moneda j, el optimo para el rango (i, j) queda de la siguente manera.

$$\text{opt}(i,j) = \text{máx}\left(\text{moneda}_i + \text{opt}_i,\ \text{moneda}_j + \text{opt}_j\right)$$

donde:

$$\text{moneda}_i\text{+opt}_i = \begin{cases} \text{monedas}[i] + \text{opt}(i+2, j) & \text{si monedas}[i+1] > \text{monedas}[j] \\ \text{monedas}[i] + \text{opt}(i+1, j-1) & \text{sino} \end{cases}$$

$$\text{moneda}_j\text{+opt}_j = \begin{cases} \text{monedas}[j] + \text{opt}(i+1, j-1) & \text{si monedas}[i] > \text{monedas}[j-1] \\ \text{monedas}[j] + \text{opt}(i, j-2) & \text{sino} \end{cases}$$

2.2. Algoritmo para obtener la reconstruccion de la solucion

Partiendo de la matriz optima, desarrollamos un algoritmo que brinde la secuencia de monedas que se deberian elegir para obtener la ganancia maxima, mediante llamadas recursivas.

1. Casos base.
   - $i < j$ El rango de monedas es invalido, finalizo la recursion y devuelvo la solucion.
   - $i = j + 1$ Quedan solo dos monedas, por lo que Sophia elige la mayor
2. Evaluacion de alternativas (esta es la parte donde analizamos que sub problema conforma la solucion del problema actual y se simula la eleccion de monedas).
   - Se calcula opt_i y opt_j de la misma manera que en el algoritmo anterior, siendo los optimos si se elije la moneda i o la moneda j, y en concecuencia que moneda elijiria Mateo (la mayor), y como estas decisiones afectan las dimensiones de nuestro problema.
   - Analizamos que optimo nos da la mayor ganancia si le sumamos la mondea correspondiente (porque es lo que el algoritmo hubiera hecho), y en base al mayor agremos el indice i o el j a la solucion, respectivametne.
   - Por ultimo vemos que moneda elije Mateo, cual es la mayor, y llamamos a la funcion recursivamente con el nuevo rango i, j conseguido luego de analizar las decisones que tomarian ambos bandos.

```
1 def solucion(opt, monedas, s, i, j):
2 if i > j:
3     return s
4
```

```
if i == j + 1:
    s.append(i if monedas[i] > monedas[j] else j)
    return s

opt_i = opt[i + 2][j] if monedas[i + 1] > monedas[j] else opt[i + 1][j - 1]
opt_j = opt[i + 1][j - 1] if monedas[i] > monedas[j - 1] else opt[i][j - 2]

if monedas[i] + opt_i > monedas[j] + opt_j:
    s.append(i)

    if monedas[i + 1] > monedas[j]:
        return solucion(opt, monedas, s, i + 2, j)
    else:
        return solucion(opt, monedas, s, i + 1, j - 1)
else:
    s.append(j)

    if monedas[i] > monedas[j - 1]:
        return solucion(opt, monedas, s, i + 1, j - 1)
    else:
        return solucion(opt, monedas, s, i, j - 2)
```

Siguendo estos pasos recursivamente obtenemos la secuencia de monedas que Sophia deberia elegir para obtener la ganancia maxima.

3. Complejidad

La complejidad del algoritmo se puede ver como la complejidad de las siguentes partes, tomando siempre N, como la cantidad de monedas.

3.1. Complejidad del algoritmo para obtener la matriz optima

El algoritmo para obtener la matriz optima, tiene la siguente complejidad.

1. Crear la matriz `opt` de $N \times N$ $\mathcal{O}(N^2)$.
2. Plantear los casos base de una moneda $\mathcal{O}(N)$.
3. Plantear los casos base de dos monedas $\mathcal{O}(N)$.
4. Completar la matriz optima $\mathcal{O}(N^2)$, recorro la matriz de $N \times N$ y hago un trabajo de $\mathcal{O}(1)$ por iteracion.

Por lo que la complejidad nos queda.

$$T(N) = \mathcal{O}(N^2 + N + N + N^2) = \mathcal{O}(N^2)$$

3.2. El algoritmo para reconstruir la solucion

El algoritmo para reconstruir la solucion tiene la siguente complejidad.

1. Trabajo por llamada recursiva. En cada llamda realiza operaciones como comparaciones, accesos a listas y operaciones arimeticas, todas con una complejidad de $\mathcal{O}(1)$.
2. Numero de llamados recursivos. En cada llamda realiza un llamado recursivo, que achica o la variable i, o la j, en una o dos unidades. Como el rango inicial es de N, a lo sumo tendremos N llamadas recursivas.

Por lo tanto la complejidad nos queda.

$$T(N) = \mathcal{O}(N)$$

3.3. Complejidad total

Finalmente la complejidad total del algoritmo, desde calcular la matriz de resultados, hasta calcular la secuencia que se debe seguir para llegar al maximo encotnrado, es la suma de ambas partes del algoritmo. Por lo que la complejidad total queda.

$$T(N) = \mathcal{O}(N^2 + N) = \mathcal{O}(N^2)$$

3.4. Efecto de los parámetros

Similar al $TP1$, podemos ver que de la misma manera, los valores de las monedas no afectan el tiempo de ejecución del algoritmo, ya que este solo evalua el valor de las monedas para llenar la tabla de optimalidad, realizando operaciones arimeticas, que asumimos se realizan en tiempo constante. Por lo que la complejidad del algoritmo esta dada solo por la cantidad de monedas y no por sus valores.

4. Seguimiento de ejemplos

4.1. Ejemplo 1

Tomemos el arreglo de monedas.

$$M = [20, 5, 10, 85, 30]$$

El algoritmo *greedy* del TP1 nos hubiera dado el siguente resultado, tomando la mayor entre las monedas disponibles.

- **Monedas de Sophia:** $[30, 20, 10] \implies \text{Ganancia} = 30 + 20 + 10 = 60$
- **Monedas de Mateo:** $[85, 5] \implies \text{Ganancia} = 85 + 5 = 90$

Si comparamos las ganancias obtenidas (60 vs. 90), se observa claramente que el enfoque *greedy* no es optimo, ya que la ganancia de Mateo resulta ser mayor a la de Sophia.

Por otro lado, con nuestro algoritmo, realizamos los siguentes pasos.

1. Creamos la matriz optima de $N \times N$

$i \setminus j$	1	2	3	4	5
1	–	–	–	–	–
2	–	–	–	–	–
3	–	–	–	–	–
4	–	–	–	–	–
5	–	–	–	–	–

2. Llenamos los casos base

$i \setminus j$	1	2	3	4	5
1	20	20	–	–	–
2	–	5	10	–	–
3	–	–	10	85	–
4	–	–	–	85	85
5	–	–	–	–	30

3. Terminamos de completar la matriz segun nos indica el algoritmo

$i \setminus j$	1	2	3	4	5
1	20	20	25	95	110
2	–	5	10	90	90
3	–	–	10	85	40
4	–	–	–	85	85
5	–	–	–	–	30

Vemos entonces que nuestra nueva ganancia maxima, encontrada en la esquina superior derecha de nuestra tabla, es de 110, mayor a la de nuestro algoritmo *greedy*.Finalmente para obtener la secuencia de monedas que deberia elegir Sophia a partir de la matriz OPT ya calculada para $M = [20, 5, 10, 85, 30]$, el recorrido recursivo desde el estado inicial $(i = 1, j = 5)$ es el siguente.

1. **Estado inicial** $(i = 1, j = 5) \implies$ Rango $[20, 5, 10, 85, 30]$:
   - Si Sophia elige $M[1] = 20$: Mateo ve $M[2] = 5$ y $M[5] = 30$. Elige 30. Queda el subproblema $OPT[2][4] = 90$.
   - Si Sophia elige $M[5] = 30$: Mateo ve $M[1] = 20$ y $M[4] = 85$. Elige 85. Queda el subproblema $OPT[1][3] = 25$.
   - **Decisión**: Sophia elige 20.
   - **Moneda guardada**: $M[1] = 20$.
   - **Siguiente estado**: $(i = 2, j = 4)$.
2. **Estado** $(i = 2, j = 4) \implies$ Rango $[5, 10, 85]$:
   - Si Sophia elige $M[2] = 5$: Mateo ve $M[3] = 10$ y $M[4] = 85$. Elige 85. Queda $OPT[3][3] = 10$.
   - Si Sophia elige $M[4] = 85$: Mateo ve $M[2] = 5$ y $M[3] = 10$. Elige 10. Queda $OPT[2][2] = 5$.
   - **Decisión**: Sophia elige 85.
   - **Moneda guardada**: $M[4] = 85$.
   - **Siguiente estado**: $(i = 2, j = 3)$.
3. **Estado** $(i = 2, j = 3) \implies$ Rango $[5, 10]$ (Caso base de 2 elementos):
   - Sophia elige la moneda máxima entre $M[2] = 5$ y $M[3] = 10$.
   - **Decisión**: Sophia elige 10.
   - **Moneda guardada**: $M[3] = 10$.

Resultado final del seguimiento:

- **Monedas elegidas por Sophia (s):** $[20, 85, 10]$, con una ganancia acumulada de 115.
- **Monedas elegidas por Mateo:** $[30, 5]$, con una ganancia acumulada de 35.

4.2. Ejemplo 2

4.2.1. 10 Monedas

Tomemos el arreglo de monedas que se encuentra en el archivo 10.txt, tal que.

$$M = [520, 781, 334, 568, 706, 362, 201, 482, 19, 145]$$

Donde se nos indica que las ganancias de los hermanos deberia ser la siguente.

- Ganancia Sophia : 2338
- Ganancia Mateo : 1780

Mediante nuestro algoritmo, realizamos los siguentes pasos para hallar nuestra matriz optima, con $N = 10$.

1. Creamos la matriz optima de $N \times N$

$i \setminus j$	1	2	3	4	5	6	7	8	9	10
1	–	–	–	–	–	–	–	–	–	–
2	–	–	–	–	–	–	–	–	–	–
3	–	–	–	–	–	–	–	–	–	–
4	–	–	–	–	–	–	–	–	–	–
5	–	–	–	–	–	–	–	–	–	–
6	–	–	–	–	–	–	–	–	–	–
7	–	–	–	–	–	–	–	–	–	–
8	–	–	–	–	–	–	–	–	–	–
9	–	–	–	–	–	–	–	–	–	–
10	–	–	–	–	–	–	–	–	–	–

2. Llenamos los casos base
3. Terminamos de completar la matriz segun nos indica el algoritmo

Podemos ver que efectivamente encontramos nuestro maximo valor acumulado en la esquina superior derecha de nuestra matriz, el valor 2338, que coincide con el valor esperado. Luego si queremes encontrar el valor acumulada de Mateo, no basta con sumar los valores del arreglo original, dandos asi el valor total de las monedas, y luego restando la gnancia obtenida por Sophia.

$$\text{Ganancia de Mateo} = \sum_{k=1}^{n} M[k] - \text{Ganancia de Sophia} = 4118 - 2338 = 1780$$

$i \setminus j$	1	2	3	4	5	6	7	8	9	10
1	520	781	—	—	—	—	—	—	—	—
2	—	781	781	—	—	—	—	—	—	—
3	—	—	334	568	—	—	—	—	—	—
4	—	—	—	568	706	—	—	—	—	—
5	—	—	—	—	706	706	—	—	—	—
6	—	—	—	—	—	362	362	—	—	—
7	—	—	—	—	—	—	201	482	—	—
8	—	—	—	—	—	—	—	482	482	—
9	—	—	—	—	—	—	—	—	19	145
10	—	—	—	—	—	—	—	—	—	145

$i \setminus j$	1	2	3	4	5	6	7	8	9	10
1	520	781	854	1349	1560	1711	2022	2193	2041	2338
2	—	781	781	1115	1349	1821	1711	2022	2193	2212
3	—	—	334	568	1040	1040	1241	1412	1260	1667
4	—	—	—	568	706	930	930	1389	1412	1431
5	—	—	—	—	706	706	907	1068	926	1333
6	—	—	—	—	—	362	362	683	844	863
7	—	—	—	—	—	—	201	482	220	627
8	—	—	—	—	—	—	—	482	482	501
9	—	—	—	—	—	—	—	—	19	145
10	—	—	—	—	—	—	—	—	—	145

Podemos ver que nuestro algoritmo encontro la solucion optima ya que la ganancia de Sophia es mayor a la de Mateo, $4118 > 1780$.

4.2.2. Reconstruccion de la solucion

Una vez tenemos la matriz *opt*, nos queda recontruir la solucion como hizimos en el ejemplo anterior, para encontrar la secuencia de monedas que debe elegir Sophia en cada turno. Vemos que nos queda que Sophia debe elegir las siguentes Monedas.

$$\text{Monedas Sophia} = M = [145, 781, 568, 362, 482]$$

4.3. Ejemplo 3

4.3.1. Caso de empate con el algoritmo greedy

5. Ajuste por cuadrados minimos

5.1. Hipotesis

Para verificar que el algoritmo tiene un tiempo de ejecución de $O(N^2)$, realizamos un ajuste de cuadrados con la siguiente función.

$F(N) = a * N^2 + b$

$$F(N) = a \times N^2 + b$$

donde a y b, son los parámetros a ajustar.

5.2. Datos de entrada

- N, el tamaño del problema (la cantidad de monedas)
- $T(N)$ el tiempo de ejecución para cada cada tamaño N

5.3. Recopilacion de datos

Para obtener mediciones confiables y reducir el ruido, seguimos los siguientes pasos:

- Múltiples tamaños. Probamos con 20 tamaños distintos, espaciados uniformemente entre $n = 100$ y $n = 1{,}000$ (podemos ver que a diferencia del tp anterior, ahora nuestro rango es mucho menor, esto ya que el algoritmo anterior era lineal y este polinomial).
- Múltiples datasets. Para cada tamaño n, generamos 10 conjuntos de datos aleatorios independientes
- Promedio de mediciones. El tiempo de ejecución reportado para cada tamaño n es el promedio aritmético de las mediciones obtenidas para ese tamaño.

5.4. Codigo usado

```
from util import time_algorithm
from TP2 import elegir

# Siempre seteamos la seed de aleatoridad para que los resultados
    sean reproducibles
```

```
seed(12345)
np.random.seed(12345)

sns.set_theme()

def get_random_array(size: int):
    return np.random.randint(0, 100.000, size)

def main():
    # La variable x van a ser los valores del eje x de los graficos en todo el notebook
    # Tamanio minimo=100, tamanio maximo=10kk, cantidad de puntos=20
    x = np.linspace(100, 1000, 20).astype(int)

    results = time_algorithm(elegir, x, lambda s: [get_random_array(s)])

    ax: plt.Axes
    fig, ax = plt.subplots()
    ax.plot(x, [results[i] for i in x], label="Medicion")
    ax.set_title('Tiempo de ejecucion de elegir')
    ax.set_xlabel('Tamanio del array')
    ax.set_ylabel('Tiempo de ejecucion (s)')

    # scipy nos pide una funcion que recibe primero x y luego los parametros a ajustar:
    f = lambda x, c1, c2: c1 * (x ** 2) + c2

    c, pcov = sp.optimize.curve_fit(f, x, [results[n] for n in x])

    print(f"c_1 = {c[0]}, c_2 = {c[1]}")
    r = np.sum((c[0] * (x ** 2) + c[1] - [results[n] for n in x])**2)
    print(f"Error cuadratico total: {r}")

    y_medido = np.array([results[n] for n in x])
    y_predicho = c[0] * (x ** 2) + c[1]

    ss_res = np.sum((y_medido - y_predicho)**2)
    ss_tot = np.sum((y_medido - np.mean(y_medido))**2)

    r_cuadrado = 1 - (ss_res / ss_tot)
    print(f"R^2 = {r_cuadrado}")


    ax.plot(x, [c[0] * (n ** 2) + c[1] for n in x], 'r--', label="Ajuste")
    ax.legend()

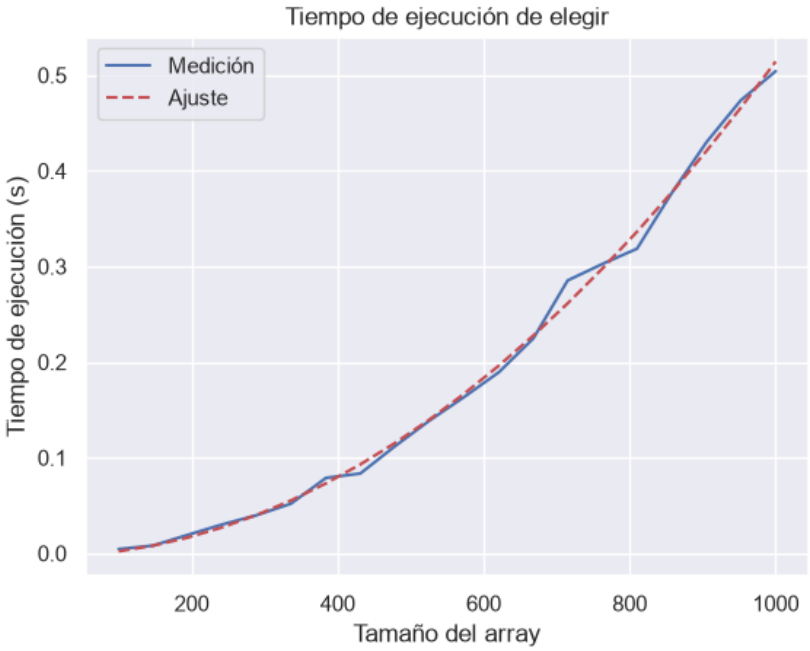
    plt.show()


if __name__ == '__main__':
    main()
```

5.5. Resultados

Los resultados del ajuste por cuadrado mínimo son.

- $a = 5{,}174732735449362e-07$
- $b = -0{,}0031624210643063184$
- Error cuadrático total $(R) = 0{,}001397511937062671$
- Calidad del ajuste $(R^2) = 0{,}9972484750254462$



5.6. Interpretacion

Usamos el coeficiente de determinación R^2

- $R^2 = 1{,}00$: Ajuste perfecto
- R^2 mayor 0,98: Excelente ajuste → Complejidad verificada
- R^2 mayor 0,95: Muy buen ajuste → Complejidad muy probable
- R^2 menor 0,90: Ajuste pobre → Resultados no concluyentes

5.7. Conclusion

Con un valor de $R^2 = 0{,}99$ (mejor que el anterior), concluimos que la complejidad del algoritmo es $O(N^2)$, debido a la alta calidad del ajuste.

6. Conclusiones